

HOTEL MANAGEMENT SYSTEM

CS23333 – OBJECT ORIENTED PROGRAMMING USING JAVA PROJECT REPORT

Submitted by

M REHANA FATHIMA – 241001127

JANANI R – 241001090

OF

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



RAJALAKSHMI ENGINEERING COLLEGE
BONAFIDE CERTIFICATE

Certified that this project titled “Hotel Management System” is the bonafide work of “**M REHANA FATHIMA (241001127), JANANI R (241001090)**” who carried out the project work under my supervision.

SIGNATURE

Dr.Valarmathie

HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai – 602105

SIGNATURE

Mrs.Usha S

COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai - 602105

This project is submitted for CS23333 – Object Oriented Programming using
Java held on _____

INTERNAL EXAMINAR

EXTERNAL EXAMINAR

TABLE OF CONTENTS

S.No	Title	Page No
1.1	Abstract	4
1.2	Introduction	4
1.3	Purpose	4
1.4	Scope of the project	5
1.5	Software requirement specification	6
2.1	Use case diagram	7
2.2	Entity-relationship diagram	7
2.3	Data flow diagram	8
3	Module description	8
4	Implementation	11
5	Conclusion	18
6	References	18

1 HOTEL MANAGEMENT SYSTEM

1.1 ABSTRACT

The Hotel Management System is a Java-based application designed to simplify and automate hotel operations such as room booking, customer management and staff administration. The system uses Java as the front-end technology and MySQL as the back-end database for secure and efficient data storage. Its main objective is to provide a centralized platform that minimizes manual effort, reduces errors, and improves operational efficiency in hotel management.

The integration of MySQL ensures reliable data handling, quick retrieval, and data integrity across all modules. With a user-friendly interface, modular design, and database-driven architecture, this project demonstrates how Java and MySQL can be effectively combined to create a robust, scalable, and efficient hotel management solution.

1.2 INTRODUCTION

The Hotel Management System is a Java-based application that uses MySQL as its database to automate and simplify hotel operations. It manages key tasks such as room booking, customer registration and check-in/check-out processes. The system reduces manual work, minimizes errors, and ensures efficient handling of guest information and room availability. With its user-friendly interface and secure database connectivity, it allows hotel staff to manage daily activities more effectively. This project aims to enhance accuracy, speed, and reliability in hotel management while providing a scalable and efficient solution for modern hospitality needs.

1.3 PURPOSE

The purpose of the Hotel Management System is to automate hotel operations and improve overall management efficiency. It helps reduce manual paperwork, eliminate data redundancy, and ensure accurate handling of guest records, room availability, and manage employees. The system provides a digital platform for both administrators and staff to manage hotel activities quickly and effectively.

1.4 SCOPE OF THE PROJECT

This project covers key hotel functions including room booking, customer registration, check-in/check-out management, staff details, and billing. It can be implemented in small to medium-sized hotels to streamline daily operations, enhance data security, and provide a reliable foundation for future upgrades and online booking features.

1.5 SOFTWARE REQUIREMENTS OF THE PROJECT

References

Java Swing Tutorial – javatpoint

Java JDBC Tutorial – javatpoint

Overall Description

The Hotel Management System provides authorized users with easy access to hotel records, streamlining reservations, billing, and guest management. It simplifies hotel operations, ensuring accurate records, faster check-ins/check-outs, and improved customer satisfaction.

Product Perspective

The system uses a **client-server architecture** and is compatible with Microsoft Windows OS. The front-end is developed using **Java Swing/AWT**, while **MySQL** handles the back-end database management, ensuring reliable and efficient data processing.

Product Functionality

- a) **Admin Login** – Enables secure administrator login.
- b) **Add Room** – Facilitates adding new room details and availability.
- c) **Add Employee** – Facilitates adding new employee details and availability.
- d) **Add Drivers** – Facilitates adding new drivers and their details.
- e) **Search Room** – Allows viewing and updating room information.
- f) **New Customer Form** – Supports creation of new room reservations.
- g) **Update Room** – Allows viewing and modifying existing bookings.
- h) **Check-In/Check-Out** – Manages guest check-in and check-out processes.
- i) **Customer Info** – View customer info and edit or update them.

Hardware Requirements:

- Processor: Intel i3 or higher
- RAM: 4GB or higher
- Hard Disk: 500GB or higher
- OS: Windows 8, 10, 11

Software Requirements:

- Database: MySQL
- Frontend: Java (Swing/AWT)
- Technology: Java (JDBC)
- Xampp Software

Assumptions and Dependencies

- **Administrators will create and securely share login credentials.**
- **The system depends on the MySQL database for storing all hotel-related data.**

Functional Requirements

1. Login Module (LM)

Admins log in with username and password.

Passwords are masked for security.

Access is granted only after database verification.

2. Room Management Module (RMM)

Add, view, update, or delete room details.

Update room availability in real-time.

3. Booking Module (BM)

Create, view, or modify reservations.

Assign rooms based on availability.

4. Check-In/Check-Out Module (CICM)

Manage guest check-in and check-out.

Update room status automatically.

5. Billing Module (BIM)

Calculate and generate invoices for guests.

Include room charges, taxes, and additional services.

6. Server Module (SM)

Handles database communication and ensures data consistency.

Validates and executes requests from other modules.

2. SYSTEM FLOW DIAGRAMS

2.1 USE CASE DIAGRAMS

Admin (1) —————< (Many) Employee

|

└—————< (Many) Room

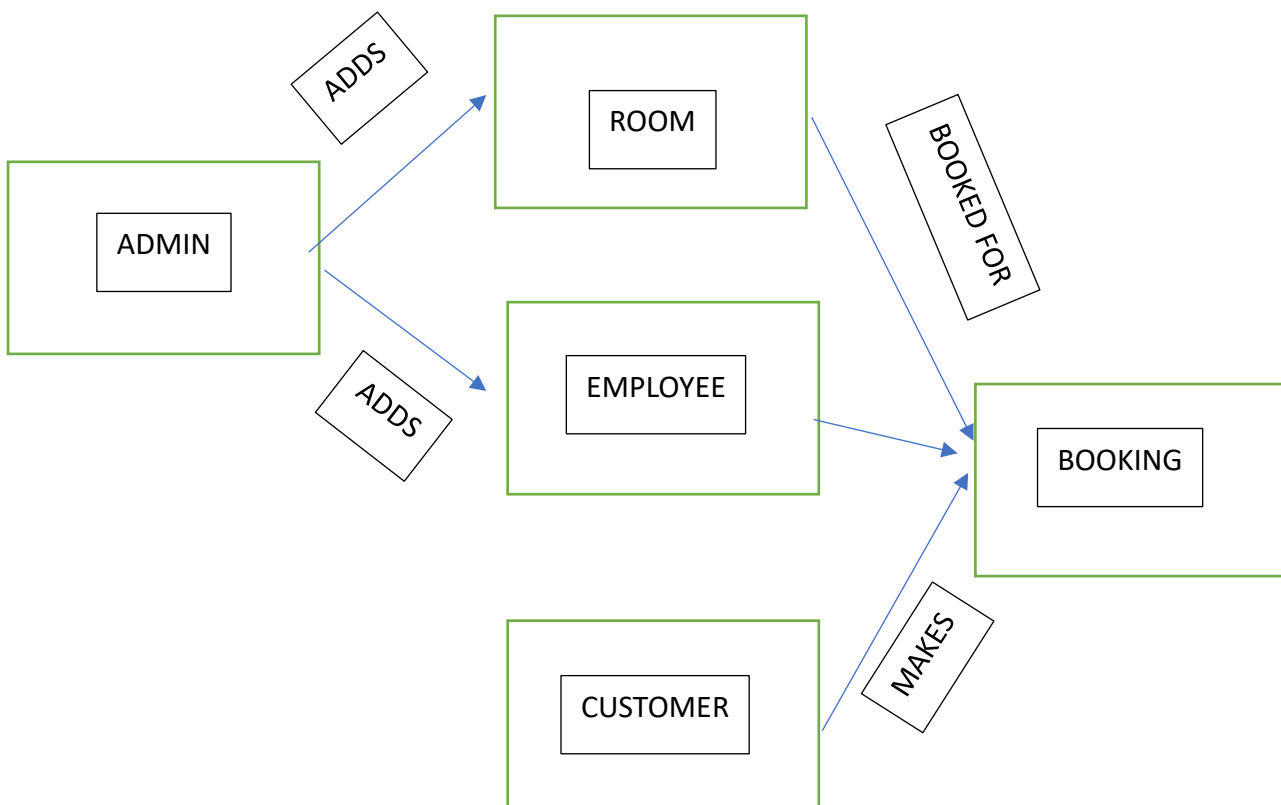
Employee (1) —————< (Many) Booking >————— (1)

Customer

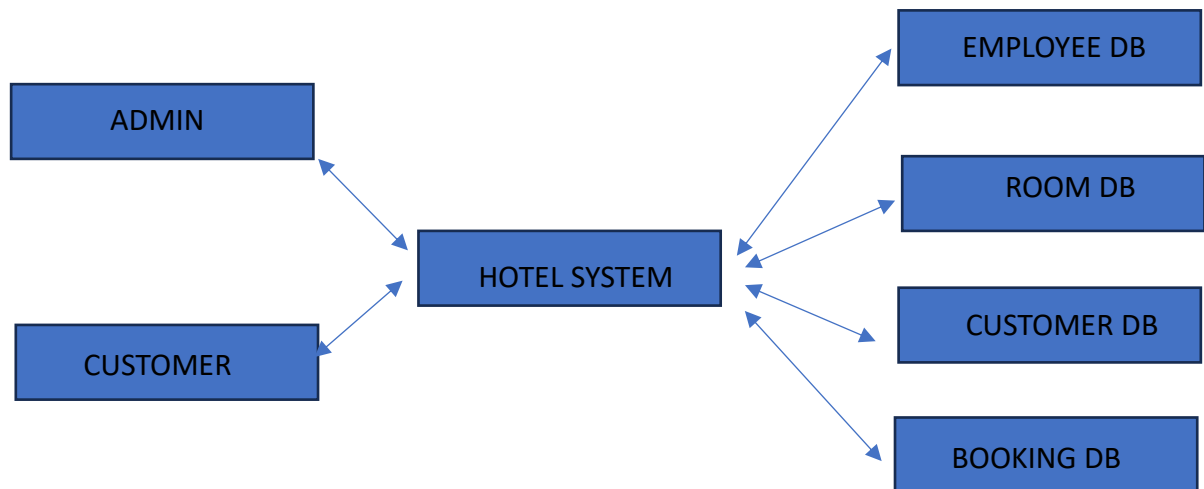
|

└————— (1) Room

2.2 ENTITY RELATIONSHIP DIAGRAMS



2.3 DATA FLOW DIAGRAMS



3. MODULE DESCRIPTION

ADMIN LOGIN

Admin can login with username and password provided by the administration.

ADD ROOM, ADD EMPLOYEES, ADD DRIVERS

Admin can add extra rooms and newly employed employees and also drivers and also remove them from the respective databases.

NEW CUSTOMER FORM

As customers check-in they can be added into the customer database using this option. They are added after certain details about them are inserted in the form.

SEARCH ROOM

Admin can search for rooms which are currently occupied and are empty and can assign them to the customers accordingly. They can also state if the room is currently cleaned or not cleaned.

CHECK-IN AND CHECK-OUT FORMS

Admin can update the status of rooms and once the customer is checked in they get added to the customer database and once checked out they are removed from the customer database.

LOGOUT

Once the admin is done updating the database they can log out of the application with the administration provided username and password.

4. IMPLEMENTATION

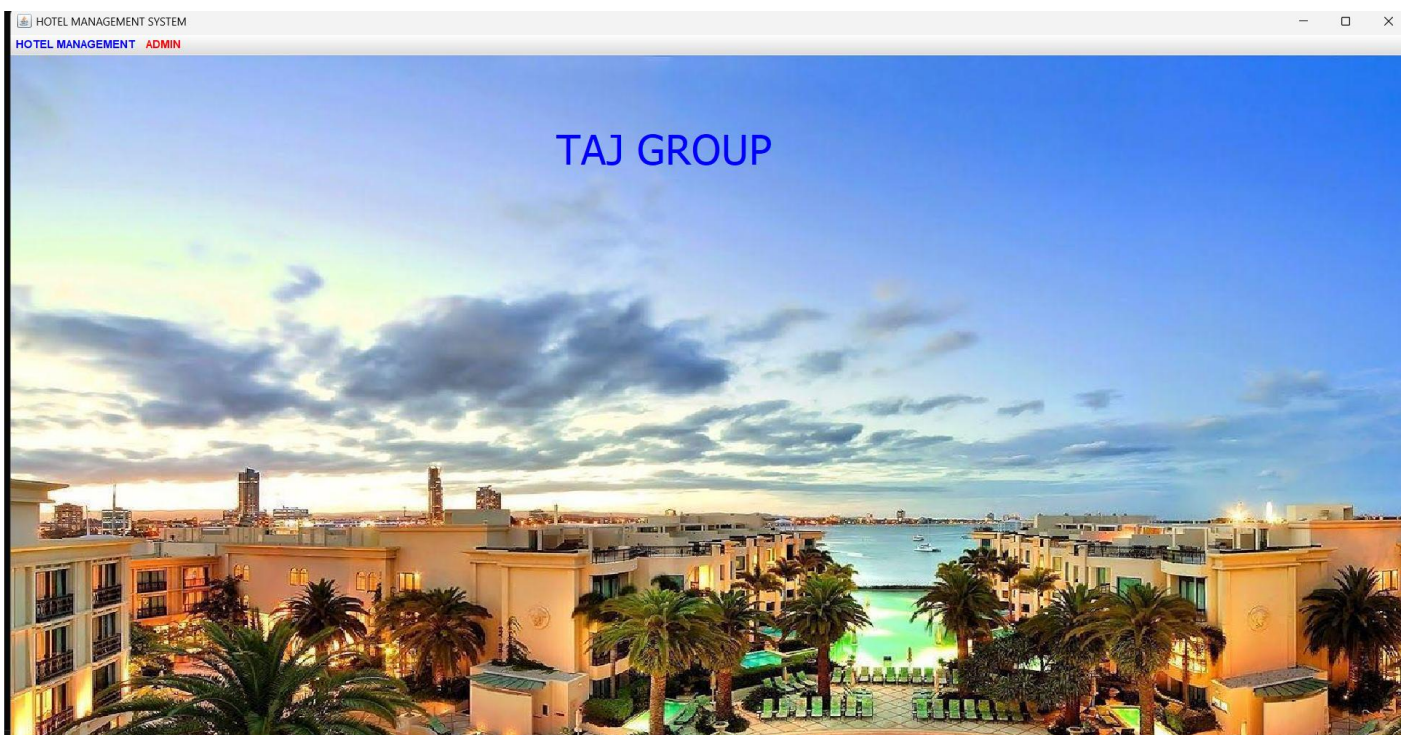
Username

Password

Login

Cancel





New Customer Form

Room

Department

All Employee Info

Customer Info

Manager Info

Check Out

Update Check Status

Update Room Status

Pick up Service

Search Room

Log Out



NEW CUSTOMER FORM

ID :

Passport

Number :

Name :

Gender :

☐ Male ☐ Female

Country :

Allocated Room Number :

101

Checked-In :

Deposit :

Add

Back



Room Number

Availability

Clean Status

Price

Bed Type

101

Occupied

Cleaned

100

Single Bed

101

Occupied

Cleaned

1000

Single Bed

102

Occupied

Cleaned

1000

Single Bed

103

Occupied

Cleaned

1000

Single Bed

Load Data

Back



Name

Age

Gender

Job

Salary

Phone

Aadhar

Gmail

sakthi

19

female

Front Desk Clerks

2000

68669

242547

shakthi@g.ailljh

pooja

19

female

Front Desk Clerks

1000

9025005348

7773641444647

pooja@pooja.com

rehana

25

female

Manager

1000

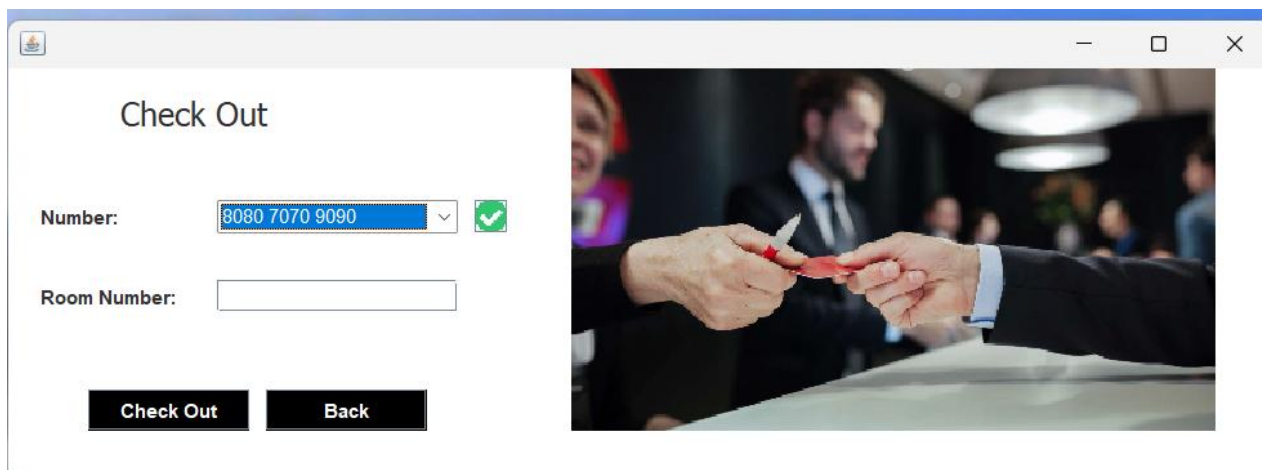
9245587746

773641444646

rehana@rehana.com

Load Data

Back



The screenshot shows a web application window titled "Check Out". On the left, there is a form with two input fields: "Number:" with a dropdown menu showing "8080 7070 9090" and a green checkmark icon, and "Room Number:" with an empty text box. Below these fields are two buttons: "Check Out" and "Back". On the right side of the window, there is a background image of a hotel lobby where a person in a red uniform is handing a red card to a guest.

4.2 DATABASE DESIGN

The data in the system has to be stored and retrieved from database. Designing the database is part of system design. Data elements and data structures to be stored have been identified at analysis stage. They are structured and put together to design the data storage and retrieval system.

A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and efficiently. The general objective is to make database access easy, quick, inexpensive and flexible for the user. Relationships are established between the data items and unnecessary data items are removed. Normalization is done to get an internal consistency of data and to have minimum redundancy and maximum stability. This ensures minimizing data storage required, minimizing chances of data inconsistencies and optimizing for updates. The MySQL database has been chosen for developing the relevant databases.

4.3 CODE

1. JAVA LOGIN PAGE CODE

2. JDBC CODE

3. MYSQL CODE

1. JAVA LOGIN PAGE CODE

```
package hotel.management.system;

import java.awt.*;
import javax.swing.*;
import java.awt.event.*;
import java.sql.*;

public class Login extends JFrame implements ActionListener{

    JLabel l1,l2;
    JTextField t1;
    JPasswordField t2;
    JButton b1,b2;

    Login(){

        super("Login");

        setLayout(null);

        l1 = new JLabel("Username");
        l1.setBounds(40,20,100,30);
        add(l1);

        l2 = new JLabel("Password");
        l2.setBounds(40,70,100,30);
        add(l2);

        t1=new JTextField();
        t1.setBounds(150,20,150,30);
        add(t1);

        t2=new JPasswordField();
        t2.setBounds(150,70,150,30);
        add(t2);

        ImageIcon i1 = new
ImageIcon(ClassLoader.getResource("hotel/management/system/icons/second.jpg"))
;

        Image i2 = i1.getImage().getScaledInstance(200,200,Image.SCALE_DEFAULT);
        ImageIcon i3 = new ImageIcon(i2);
        JLabel l3 = new JLabel(i3);
        l3.setBounds(350,10,150,150);
        add(l3);

        b1 = new JButton("Login");
        b1.setBounds(40,140,120,30);
        b1.setFont(new Font("serif",Font.BOLD,15));
        b1.addActionListener(this);
```

```

•         b1.setBackground(Color.BLACK);
•         b1.setForeground(Color.WHITE);
•         add(b1);
•
•         b2=new JButton("Cancel");
•         b2.setBounds(180,140,120,30);
•         b2.setFont(new Font("serif",Font.BOLD,15));
•         b2.setBackground(Color.BLACK);
•         b2.setForeground(Color.WHITE);
•         add(b2);
•
•         b2.addActionListener(this);
•
•
•         getContentPane().setBackground(Color.WHITE);
•
•         setVisible(true);
•         setSize(600,300);
•         setLocation(600,350);
•
•     }
•
•     public void actionPerformed(ActionEvent ae){
•         if(ae.getSource()==b1){
•             try{
•                 conn c1 = new conn();
•                 String u = t1.getText();
•                 String v = t2.getText();
•
•                 String q = "select * from login where username='"+u+"' and
password='"+v+"'";
•
•                 ResultSet rs = c1.s.executeQuery(q);
•                 if(rs.next()){
•                     new Dashboard().setVisible(true);
•                     setVisible(false);
•                 }else{
•                     JOptionPane.showMessageDialog(null, "Invalid login");
•                     setVisible(false);
•                 }
•             }catch(Exception e){
•                 e.printStackTrace();
•             }
•             }else if(ae.getSource()==b2){
•                 System.exit(0);
•             }
•         }
•     }
•     public static void main(String[] arg){
•         new Login();
•     }
• }

```

2. JDBC CODE

```
package hotel.management.system;

import java.sql.*;
import javax.swing.JOptionPane;

public class conn {
    Connection c;
    Statement s;
    public conn() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            c =
DriverManager.getConnection("jdbc:mysql://localhost:3306/hotelmanagement_system","root",""
);
            c.setAutoCommit(true);
            s = c.createStatement();
        } catch (Exception e) {
            JOptionPane.showMessageDialog(null, e);
        }
    }
}
```

3. MYSQL CODE

```
2. create database hotelmanagementsystem;
3.
4. show databases;
5.
6. use hotelmanagementsystem;
7.
8. create table login(username varchar(25), password varchar(25));
9.
10.insert into login values('admin', '12345');
11.
12.select * from login;
13.
14.create table employee(name varchar(25), age varchar(10), gender varchar(15), job
    varchar(30), salary varchar(15), phone varchar(15), email varchar(40), aadhar
    varchar(20));
15.
16.describe employee;
17.
18.select * from employee;
19.
20.create table room(roomnumber varchar(10), availability varchar(20), cleaning_status
    varchar(20), price varchar(20), bed_type varchar(20));
21.
22.select * from room;
23.
```



```

24.update room set availability = 'Available' where roomnumber = '101';
25.
26.create table driver(name varchar(20), age varchar(10), gender varchar(15), company
   varchar(20), branch varchar(20), available varchar(20), location varchar(40));
27.
28.select * from driver;
29.
30.ALTER TABLE driver RENAME COLUMN branch TO brand;
31.
32.create table customer(document varchar(20), number varchar(30), name varchar(30),
   gender varchar(15), country varchar(20), room varchar(10), checkintime varchar(80),
   deposit varchar(20));
33.
34.select * from customer;
35.
36.create table department(department varchar(30), budget varchar(30));
37.
38.insert into department values('Front Office','500000');
39.insert into department values('Housekeeping', '40000');
40.insert into department values('Food and Beverage', '23000');
41.insert into department values('Kitchen or Food Production', '540000');
42.insert into department values('Security', '320000');
43.
44.select * from department;

```

5. CONCLUSION

The Hotel Management System developed using Java and MySQL successfully streamlines the management of hotel operations, including room allocation, employee management, and customer check-in/check-out processes. By integrating a secure database and a user-friendly interface, the system enhances efficiency, minimizes manual errors, and ensures real-time data handling. Overall, this project demonstrates the effective application of Java for backend logic and MySQL for reliable data storage, resulting in a scalable and efficient solution for hotel administration.

6. REFERENCES

- [W3Schools – Java and MySQL Tutorials](#)
- GeeksforGeeks – Java Database Connectivity (JDBC)

TutorialsPoint, *Java MySQL Integration Examples*